



BULUT BİLİŞİM

Doç. Dr. Eftal ŞEHİRLİ

Hafta-5

Bulut Bilişim Mimarisi ve Bileşenleri

Konu Başlıkları

1. Ön Uç (Client Side)
 1. Kullanıcı cihazları
 2. Arayüzler
 3. Uygulama katmanı
2. Arka Uç (Server Side)
 1. Sunucular
 2. Veritabanları
 3. Depolama sistemleri
3. Yönetim ve Orkestrasyon
 1. Kaynak izleme
 2. Yük dengeleme
 3. Otomasyon
4. API ve Servis Arabirimleri
 1. REST API
 2. Entegrasyon
5. SLA (Service Level Agreement)
 1. Tanım
 2. Önemi
 3. Ölçütler

Bulut Mimarisi Nedir?

- Sunucular, depolama, veritabanları ve ağ bileşenleri gibi bulut bilişim kaynaklarının, internet üzerinden esnek ve ölçeklenebilir hizmetler sunmak amacıyla yapılandırılması, organize edilmesi ve yönetilmesi sürecidir.
- Ön uç + Arka uç = Temel yapı
- Ölçeklenebilirlik
- Esneklik
- Güvenlik
- Performans
- Büyük sağlayıcılar: AWS, Azure, GCP



1. Ön Uç (Client Side)

Ön Uç Tanımı

- Kullanıcıya en yakın katman
- Erişim noktası
- Kullanıcı deneyiminin başladığı alan
- Cihazlar + arayüzler + uygulama katmanı

1. Ön Uç (Client Side)

Kullanıcı Cihazları

Cihaz Türleri:

- PC / Desktop
- Mobil cihazlar (Android/iOS)
- Tabletler
- Akıllı TV – IoT
- Tarayıcı tabanlı istemciler

Anahtar Kelime: “Thin client – ince istemci mantığı”

1. Ön Uç (Client Side)

Kullanıcı Cihazları

Kullanıcı Cihazlarının Rolü:

- İstek gönderme
- Sonuç görüntüleme
- Kimlik doğrulama (Çok faktörlü kimlik doğrulama (MFA), token)
- Oturum yönetimi
- Basit önbellekleme

1. Ön Uç (Client Side)

Kullanıcı Cihazları

Ön Uç Önemi :

- Platform bağımsız erişim
- Çoklu cihaz desteği
- Her yerden erişim
- Responsive arayüz
- Cihazda yük yok → işlem bulutta



1. Ön Uç (Client Side)

Arayüzler

Arayüz Türleri :

- Web arayüzleri (AWS, Azure Portal)
- Mobil uygulamalar
- Komut Satırı (CLI) araçları (AWS CLI, Azure CLI, gcloud)
- Secure Shell (SSH) / Remote Desktop Protocol (RDP)

1. Ön Uç (Client Side)

Arayüzler

Arayüz Özellikleri :

- Kullanıcı dostu tasarım
- Geri bildirim
- Güvenli giriş (OAuth, MFA, token)
- Erişim kontrolü (RBAC)
- API tabanlı çalışma



1. Ön Uç (Client Side)

Uygulama Katmanı

- SaaS uygulamaları
- Web depolama
- Veritabanı yönetim sistemleri
- Bulut tabanlı IDE'ler

Görev:

→ Kullanıcı isteği → API → Arka uç



2. Arka Uç (Server Side)

Arka Uç Tanımı

- Bulutun kalbi
- Hesaplama
- Veritabanı
- Depolama
- Güvenlik + performans + ölçeklenme



2. Arka Uç (Server Side)

Sunucular

Sunucu Türleri:

- Fiziksel sunucu (bare-metal)
- Sanal makineler (EC2, Azure VM)
- Konteynerler (Docker, Kubernetes)
- Serverless (Lambda, Cloud Functions)



2. Arka Uç (Server Side)

Sunucular

Sunucuların Görevleri:

- Uygulama çalıştırma
- API isteklerini işleme
- Hesaplama
- Yük paylaşımı
- Güvenlik yönetimi



2. Arka Uç (Server Side)

Veritabanları

Türler :

- SQL (RDS, Cloud SQL)
- NoSQL (MongoDB, DynamoDB)
- Bellek tabanlı (Redis)



2. Arka Uç (Server Side)

Veritabanları

Bulut özellikleri:

- Otomatik yedek
- Replikasyon
- Failover
- Şifreleme
- Ölçeklenebilirlik



2. Arka Uç (Server Side)

Depolama Sistemleri

Türler :

- Nesne depolama (S3, Blob)
 - Key-value ilişkisi
- Dosya depolama (EFS, Filestore)
 - Dosya direkt gönderme sistemi
- Blok depolama (EBS, Persistent Disk)
 - Disk olarak hareket etme



2. Arka Uç (Server Side)

Depolama Sistemleri

Avantajlar :

- Sınırsız ölçek
- Dayanıklılık
- Maliyet

3. Yönetim ve Orkestrasyon

Yönetim/Orkestrasyon Tanım

- İzleme
- Yük dengeleme
- Otomasyon
- Kaynakların otomatik yönetimi
- Büyük sistemlerde zorunlu yapı



3. Yönetim ve Orkestrasyon

Kaynak İzleme

Neden izleme?

- Performans
- Kesinti önleme
- Güvenlik tespiti
- Maliyet optimizasyonu
- Uygulama davranışı

Araçlar: CloudWatch, Azure Monitor, GCP Operations



3. Yönetim ve Orkestrasyon

Kaynak İzleme

İzleme Metriği :

- CPU, RAM
- Ağ trafiği
- DB sorgu süresi
- API gecikmesi
- Log yönetimi



3. Yönetim ve Orkestrasyon

Yük Dengeleme (Load Balancing)

Amaç :

- Trafiği sunucular arasında dengeli dağıtmak
- Erişilebilirlik
- Performans
- Otomatik ölçeklenme



3. Yönetim ve Orkestrasyon

Yük Dengeleme

Health Check :

- Sunucu durumu kontrolü
- Status 200 OK?
 - Client's request was successfully received, understood, and accepted by the server.
- Yanıt süresi?
 - Sender response
- Aşırı yük?

Kötü durumda sunucu havuzdan çıkar.



3. Yönetim ve Orkestrasyon

Otomasyon

Kapsam :

- Auto-scaling
- Infrastructure as Code (IaC) - (Terraform, CloudFormation)
- CI/CD
- Log & alarm yönetimi
- Güvenlik taramaları
- Yedekleme otomasyonu



3. Yönetim ve Orkestrasyon

Otomasyon

Otomasyon Avantajları :

- Maliyet azaltma
- Hızlı dağıtım
- Tutarlılık
- Güvenlik
- Felaket kurtarma



3. Yönetim ve Orkestrasyon

Otomasyon

Orkestrasyon Teknolojileri :

- Kubernetes
- Docker Swarm
- Step Functions / Logic Apps
- Workflows



4. API ve Servis Arabirimleri

API Tanım

- Uygulamaların birbiriyle konuşmasını sağlayan arayüz
- Ön uç ve arka ucu bağlar
- Tüm bulut işlemleri API ile çalışır
- Entegrasyonun temeli



4. API ve Servis Arabirimleri

REST API

- Stateless
 - Önceki istekleri hatırlamama
 - İsteklerin her ayrıntıyı içermesi durumu
- HTTP metotları: GET, POST, PUT, DELETE
- JSON/XML yapısı

4. API ve Servis Arabirimleri

REST API

Kullanım Örnekleri

- Sunucu oluşturma
- Depolama alanı oluşturma
- Dosya yükleme
- Kullanıcı yönetimi
- Veritabanı erişimi
- Log kaydı açma



4. API ve Servis Arabirimleri

REST API

Kullanım Örnekleri

- 1- Kullanıcı veya uygulama bir işlem ister. Ör: Yeni bir VM oluşturmak istiyor.
- 2- Bu istek API üzerinden buluta iletilir.
- 3- API isteği doğrular
 - Yetki var mı?
 - Token doğru mu?
 - Rol (RBAC) uygun mu?
- 4- İstek arka uç yönetimine iletilir.
- 5- Arka uç gerekli kaynağı oluşturur. Ör: VM, disk, load balancer vb.
- 6- API sonuç döndürür. JSON formatında:

```
{ "status": "success", "serverId": "abc123" }
```



4. API ve Servis Arabirimleri

Entegrasyon Türleri

- Servis–Servis
 - Bulut servisleri kendi aralarında iletişim kurar.
- Uygulama–Servis
 - Mobil/Web/IoT uygulaması vb. bulut servisine istek gönderir.
- Veri akışı tasarımları
 - Event driven structure



4. API ve Servis Arabirimleri

Entegrasyon Türleri

- Servis–Servis: Örnek1
 - S3(Simple Storage Service)’e bir fotoğraf yüklenir.
 - Bu olay Lambda fonksiyonunu tetikler.
 - Lambda resmi küçültür ve tekrar S3’e koyar.
 - Entegrasyon türü: S3 (Servis) → Lambda (Servis)
- Servis–Servis: Örnek2
 - CPU %90’ın üzerine çıkarsa CloudWatch alarm üretir.
 - Bu alarm SNS (Simple Notification Service) servisine gönderilir.
 - SNS yöneticilere e-posta / SMS gönderir.
 - Entegrasyon: CloudWatch (Servis) → SNS (Servis)



4. API ve Servis Arabirimleri

Entegrasyon Türleri

- Uygulama–Servis: Örnek1
 - Web sitesi kullanıcıdan PDF alır.
 - Web uygulaması bu dosyayı S3 API’sini kullanarak depoya yükler.
 - Entegrasyon: Web App → S3 Servisi
- Uygulama-Servis: Örnek2
 - Web tabanlı chatbot kullanıcı mesajını alır.
 - Chatbot server uygulaması veritabanına API ile kayıt açar.
 - Entegrasyon: Uygulama → DynamoDB Servisi
- Uygulama-Servis: Örnek3
 - Kartla ödeme yapılır.
 - Web sitesi Stripe API’sine istek gönderir.
 - Stripe işlemi tamamlayıp sonucu döndürür.
 - Entegrasyon: Uygulama → Stripe (Bulut Ödeme Servisi)



5. SLA

SLA Nedir?

- Hizmet seviyesi sözleşmesi (Service Level Agreement)
- Hizmetin çalışma süresi garantisi (Uptime)
 - %99.0 - %99.9 - %99.95 - %99.99 - %99.999
- Performans hedefi
 - API yanıt süresi ≤ 200 ms
 - Veri tabanı sorgu gecikmesi ≤ 20 ms
 - Depolama servisi okuma hızı ≥ 300 MB/s
- Destek seviyeleri
 - Verilme hızı
 - Yanıt süreleri
 - Teknik uzmanlık seviyesi
- Kesinti durumunda uygulanacak politikaları
 - Para iadesi, %10 geri ödeme
- Veri dayanıklılığı garantileri
 - 10,000,000 dosya yükleseniz, 10,999,999 yıl boyunca sadece 1 tanesinin kaybolma ihtimali vardır.

5. SLA

SLA Önemi

- Güvenilirlik
- Sorumluluk sınırları
- Kesinti durumunda geri ödeme
- Kullanıcı hakları

5. SLA

SLA Ölçütleri

- Uptime
- Yanıt süresi
- Veri dayanıklılığı
- Destek süresi

Genel Özet

- Ön uç: cihaz + arayüz + uygulama
- Arka uç: compute + database + storage
- Yönetim: monitoring, load balancing, automation
- API: REST, entegrasyon
- SLA: hizmet garantisi

Sonuç:

Bulut Mimarisi = Performans + Güvenlik + Esneklik + Yönetilebilirlik.



Beyin Fırtınası

- Ön uç mu, arka uç mu daha kritik? Neden?
- Serverless yapılar gelecekte klasik VM'lerin yerini alır mı?
- Yük dengeleme olmasaydı modern web uygulamaları nasıl etkilenirdi?
- API tabanlı mimari olmadan bulut sistemleri nasıl olurdu?
- İzleme (monitoring) olmadan bir bulut ortamı yönetilebilir mi?

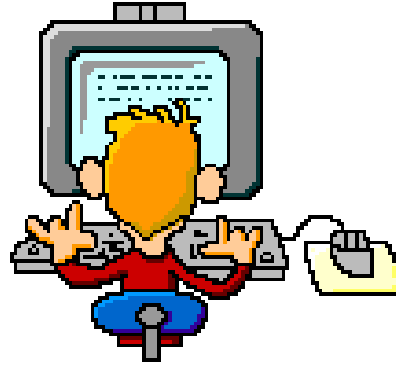


Değerlendirme Soruları

- Ön uç (client side) bileşenlerini örneklerle açıklayınız.
- Arka uç (server side) mimarisinde sunucuların temel görevleri nelerdir?
- Bulutta kullanılan veritabanı türlerini karşılaştırınız.
- Nesne depolama (object storage) ile blok depolama arasındaki farkları belirtiniz.
- Monitoring araçlarının (CloudWatch, Azure Monitor vb.) bulut yönetimindeki rolünü açıklayınız.
- Yük dengeleme (load balancing) neden yüksek erişilebilirlik için kritik bir yapıdır?
- Otomasyon (IaC, Auto Scaling, CI/CD) bulut işletme maliyetlerini nasıl etkiler?
- REST API'nin bulut mimarisindeki yeri ve temel özelliklerini tanımlayınız.
- SDK kullanımının uygulama geliştirme süreçlerine sağladığı avantajları açıklayınız.
- SLA kavramını tanımlayarak; uptime, veri dayanıklılığı ve destek seviyelerinin önemini tartışınız.



SORULAR





TEŞEKKÜRLER

Doç. Dr. Eftal ŞEHİRLİ